

Demand Forecasting EDA Checklist

8 Checks to Run Before You Train a Single Model

Supplementary material for: [66% of Demand Series Were Unforecastable... \(https://kozlov.ski/posts/demand-forecast-eda\)](https://kozlov.ski/posts/demand-forecast-eda)

How to Use This Checklist

This checklist walks you through 8 essential data quality and structure checks for demand forecasting on retail POS data. Each check includes a SQL query template (DuckDB dialect — adapt for your engine), what to look for, and how to interpret the results.

Estimated time: 30 minutes with your data loaded

Before you start: Load your POS/transaction data into a table. The queries below assume a table called `pos_data` with the schema described in the next section. Rename columns as needed.

Your Dataset Setup

Expected Schema

COLUMN	TYPE	DESCRIPTION
<code>date</code>	DATE	Transaction date
<code>store_id</code>	INT	Store identifier
<code>item_id</code>	INT	Product identifier
<code>item_brand</code>	VARCHAR	Brand name
<code>item_category</code>	VARCHAR	Product category
<code>item_unit_price</code>	DOUBLE	Listed unit price
<code>item_unit_net_price</code>	DOUBLE	Net price per unit
<code>item_discount_amount</code>	DOUBLE	Discount applied per unit
<code>item_promo_applied</code>	INT	1 if promotion active, 0 otherwise
<code>demand</code>	DOUBLE	Quantity sold (target variable)

Define Your Series Grain

Your forecasting grain is the combination of columns that uniquely identifies each row:

- **Series key:** `(date, store_id, item_id)`
- **Target:** `demand`
- **Static covariates:** store hierarchy, item hierarchy (brand, category)
- **Dynamic covariates:** `item_promo_applied`, `item_unit_net_price`

Check 1: Duplicate Series Keys

If `(date, store_id, item_id)` is your prediction grain, each combination must appear exactly once. Duplicates inflate demand counts for some products and zero out others.

☐ Verify `(date, store_id, item_id)` uniqueness

```
SELECT
  CAST(date AS DATE) AS d,
  store_id,
  item_id,
  COUNT(*) AS n_rows
FROM pos_data
GROUP BY 1, 2, 3
HAVING COUNT(*) > 1
ORDER BY n_rows DESC;
```

What to look for:

- If zero rows returned — you're clean.
- If duplicates exist, inspect a specific group to understand the cause:

```
SELECT *
FROM pos_data
WHERE store_id = <id>
  AND item_id = <id>
  AND date = '<date>'
ORDER BY demand DESC;
```

Common causes: POS export mixing sales with inventory adjustments, returns, or manual corrections. Look for rows with `demand = 0` or `unit_price = 0` alongside normal rows — these are usually non-sale records.

Check 2: Negative Demand & Discounts

Negative values in the target variable or covariates signal data quality issues upstream.

- ☐ Check for negative demand rows
- ☐ Check for negative discounts
- ☐ Check for fractional demand (expected for weight-based items?)

```
SELECT
  SUM(CASE WHEN demand < 0 THEN 1 ELSE 0 END) AS neg_demand_rows,
  SUM(CASE WHEN item_discount_amount < 0 THEN 1 ELSE 0 END)
    AS neg_discount_rows,
  AVG((demand ≠ ROUND(demand))::INT)::DOUBLE
    AS frac_non_integer_share
FROM pos_data;
```

How to interpret:

FINDING	LIKELY CAUSE	ACTION
Negative demand	Returns, voids	Investigate with POS team; exclude or aggregate
Negative discounts	Surcharges, data entry errors	Investigate; may need sign correction
Fractional demand	Weight-based items (meat, bulk)	Expected — document and proceed

If negative demand rows are < 0.1% of total, excluding them is usually safe. If they're systemic, the export query needs fixing.

Check 3: Promotions Without Discounts

If you plan to use promotion status as a covariate, verify the signal is clean.

☐ Cross-check promo flag vs. discount amount

```
SELECT
    item_promo_applied,
    COUNT(*) AS n_rows,
    AVG(item_discount_amount) AS avg_discount,
    SUM(CASE WHEN item_discount_amount = 0 THEN 1 ELSE 0 END)
        AS zero_discount_rows
FROM pos_data
GROUP BY 1;
```

How to interpret:

- Promos with discounts: expected behavior
- Promos without discounts: either (a) the promo flag is unreliable, or (b) promotions include non-price mechanisms (shelf placement, bundling, BOGO)
- Without knowing which, the feature is noisy at best and misleading at worst

Decision: If a significant share of promoted rows have zero discount, either drop the promo feature or enrich it with the promotion type from the source system.

Check 4: Pricing Identity

A simple identity should hold: `unit_price - discount ≈ net_price`. Failures indicate the pricing data can't be trusted for elasticity modeling.

☐ Verify `unit_price - discount ≈ net_price`

```
SELECT
    date,
    store_id,
    item_id,
    item_unit_price,
    item_discount_amount,
    item_unit_net_price,
    ABS(item_unit_price - item_unit_net_price
        - item_discount_amount) AS price_err
FROM pos_data
WHERE ABS(item_unit_price - item_unit_net_price
        - item_discount_amount) > 0.01
ORDER BY price_err DESC
LIMIT 20;
```

☐ Identify which stores/items are most affected

```
SELECT
    store_id,
    SUM(ABS(item_unit_price - item_unit_net_price
        - item_discount_amount)) AS total_price_err,
    COUNT(*) AS affected_rows
FROM pos_data
WHERE ABS(item_unit_price - item_unit_net_price
        - item_discount_amount) > 0.01
GROUP BY 1
ORDER BY total_price_err DESC;
```

Why it matters: If your model learns “when price drops, demand rises,” but the price data is incoherent, the model learns noise. Price elasticity estimates become meaningless.

Check 5: Within-Day Price Consistency

This check flags cases where the same item has wildly different prices at the same store on the same day.

☐ Compute min/max price ratio per day-store-item

```
WITH base AS (  
    SELECT  
        CAST(date AS DATE) AS d,  
        store_id,  
        item_id,  
        COALESCE(  
            item_unit_net_price,  
            item_unit_price - COALESCE(item_discount_amount, 0)  
        )::DOUBLE AS net_price,  
        demand::DOUBLE AS qty  
    FROM pos_data  
,  
grouped AS (  
    SELECT  
        d, store_id, item_id,  
        COUNT(*) AS n_rows,  
        MIN(net_price) AS min_price,  
        MAX(net_price) AS max_price,  
        MEDIAN(net_price) AS med_price,  
        STDDEV_POP(net_price) / NULLIF(AVG(net_price), 0)  
        AS cv_price,  
        (MAX(net_price) - MIN(net_price))  
        / NULLIF(MEDIAN(net_price), 0) AS rel_range  
    FROM base  
    GROUP BY 1, 2, 3  
)  
SELECT *  
FROM grouped  
WHERE n_rows > 1  
    AND (rel_range > 0.20 OR cv_price > 0.10)  
ORDER BY rel_range DESC  
LIMIT 20;
```

Interpretation guide:

REL_RANGE	SEVERITY	ACTION
0.05 - 0.10	Minor	Usually fine — rounding or small adjustments
0.10 - 0.20	Moderate	Review — mixed pack sizes or line-level discounts
> 0.20	High	Action needed — aggregate to volume-weighted net price

Volume-weighted net price aggregation:

```
-- Use this to collapse noisy line-level prices into clean daily prices
SELECT
  CAST(date AS DATE) AS d,
  store_id,
  item_id,
  SUM(demand) AS demand,
  SUM(net_price * demand) / NULLIF(SUM(demand), 0)
  AS vw_net_price
FROM pos_data
GROUP BY 1, 2, 3;
```

Check 6: Store Coverage

Check whether all stores report data consistently. Missing stores bias demand downward — “no data” is not “zero demand.”

- ☐ Build date spine x store spine
- ☐ Identify ghost stores (no data on most days)

```
WITH date_spine AS (  
    SELECT *  
    FROM generate_series(  
        (SELECT MIN(date) FROM pos_data),  
        (SELECT MAX(date) FROM pos_data),  
        INTERVAL 1 DAY  
    ) AS g(d)  
,  
store_spine AS (  
    SELECT DISTINCT store_id FROM pos_data  
,  
full_calendar AS (  
    SELECT ds.d, ss.store_id  
    FROM date_spine ds  
    CROSS JOIN store_spine ss  
,  
store_days AS (  
    SELECT date AS d, store_id, 1 AS has_row  
    FROM pos_data  
    GROUP BY 1, 2  
)  
SELECT  
    fc.store_id,  
    COUNT(*) AS total_days,  
    SUM(COALESCE(sd.has_row, 0)) AS active_days,  
    COUNT(*) - SUM(COALESCE(sd.has_row, 0)) AS missing_days,  
    ROUND(  
        SUM(COALESCE(sd.has_row, 0))::DOUBLE / COUNT(*), 2  
    ) AS coverage_rate  
FROM full_calendar fc  
LEFT JOIN store_days sd USING (d, store_id)  
GROUP BY 1  
ORDER BY coverage_rate ASC;
```

What to look for:

- Stores with coverage < 50% are likely “ghost stores” — unreliable POS reporting
- These stores bias demand downward if included in training

- **"No data" and "zero demand" are fundamentally different** — one means the store was closed or the system was down, the other means products were available and nobody bought them

Decision: Exclude stores below your coverage threshold from training data, or flag them for investigation with the operations team.

Check 7: Intermittency Classification

Compute how often each store-item pair actually sells within its listing window, then classify using the Syntetos-Boylan Classification (SBC).

Step 1: Compute active windows and zero share

- ☐ Compute active windows (first_seen to last_seen)
- ☐ Calculate zero share per window

```
WITH daily AS (  
    SELECT CAST(date AS DATE) AS d, store_id, item_id  
    FROM pos_data  
    GROUP BY 1, 2, 3  
)  
active_windows AS (  
    SELECT  
        store_id,  
        item_id,  
        MIN(d) AS first_seen,  
        MAX(d) AS last_seen,  
        COUNT(*) AS n_days_with_sales,  
        DATEDIFF('day', MIN(d), MAX(d)) + 1 AS lifespan_days  
    FROM daily  
    GROUP BY 1, 2  
)  
SELECT  
    store_id,  
    item_id,  
    lifespan_days,  
    n_days_with_sales,  
    1.0 - (n_days_with_sales::DOUBLE / lifespan_days)  
        AS zero_share,  
    ROUND(n_days_with_sales::DOUBLE / lifespan_days, 3)  
        AS activity_rate  
FROM active_windows  
ORDER BY zero_share DESC;
```

Step 2: Apply SBC classification (ADI + CV²)

- ☐ Compute ADI and CV² per store-item pair

```
WITH daily_demand AS (  
    SELECT
```

```

        CAST(date AS DATE) AS d,
        store_id,
        item_id,
        SUM(demand) AS daily_demand
    FROM pos_data
    GROUP BY 1, 2, 3
),
nonzero AS (
    SELECT
        store_id, item_id, d, daily_demand,
        LAG(d) OVER (
            PARTITION BY store_id, item_id ORDER BY d
        ) AS prev_d
    FROM daily_demand
    WHERE daily_demand > 0
),
intervals AS (
    SELECT
        store_id, item_id,
        daily_demand,
        DATEDIFF('day', prev_d, d) AS inter_demand_interval
    FROM nonzero
    WHERE prev_d IS NOT NULL
),
sbc_metrics AS (
    SELECT
        store_id,
        item_id,
        AVG(inter_demand_interval) AS adi,
        POWER(
            STDDEV_POP(daily_demand)
            / NULLIF(AVG(daily_demand), 0),
            2
        ) AS cv2
    FROM intervals
    GROUP BY 1, 2
    HAVING COUNT(*) ≥ 2
)
SELECT
    store_id,
    item_id,
    ROUND(adi, 2) AS adi,
    ROUND(cv2, 4) AS cv2,
    CASE
        WHEN adi ≤ 1.32 AND cv2 ≤ 0.49 THEN 'Smooth'
        WHEN adi ≤ 1.32 AND cv2 > 0.49 THEN 'Erratic'
        WHEN adi > 1.32 AND cv2 ≤ 0.49 THEN 'Intermittent'
        ELSE 'Lumpy'
    END AS sbc_class
FROM sbc_metrics;

```

SBC Quadrant Interpretation

QUADRANT	ADI	CV ²	MEANING	BEST METHOD
Smooth	≤ 1.32	≤ 0.49	Sells often, consistent qty	ETS, ARIMA
Erratic	≤ 1.32	> 0.49	Sells often, variable qty	ARIMA with caution
Intermittent	> 1.32	≤ 0.49	Sells rarely, consistent qty	Croston's
Lumpy	> 1.32	> 0.49	Sells rarely, variable qty	SBA, ADIDA

Threshold values (ADI = 1.32, CV² = 0.49) come from Syntetos & Boylan (2005). These are the standard cutoffs used in the operations research literature.

Check 8: Pareto Analysis

Build a Lorenz curve to understand demand concentration. Most retail datasets follow the 80/20 rule — a small fraction of items drives most volume.

- ☐ Build Lorenz curve of demand volume
- ☐ Identify A/B/C tiers

```
WITH item_volumes AS (  
    SELECT  
        store_id,  
        item_id,  
        SUM(demand) AS total_demand  
    FROM pos_data  
    GROUP BY 1, 2  
)  
ranked AS (  
    SELECT  
        store_id,  
        item_id,  
        total_demand,  
        SUM(total_demand) OVER (ORDER BY total_demand DESC)  
            AS cum_demand,  
        SUM(total_demand) OVER () AS grand_total,  
        ROW_NUMBER() OVER (ORDER BY total_demand DESC) AS rn,  
        COUNT(*) OVER () AS total_items  
    FROM item_volumes  
)  
SELECT  
    rn,  
    ROUND(rn::DOUBLE / total_items, 4) AS pct_items,  
    ROUND(cum_demand::DOUBLE / grand_total, 4) AS pct_demand,  
    CASE  
        WHEN cum_demand::DOUBLE / grand_total ≤ 0.80  
            THEN 'A-tier'  
        WHEN cum_demand::DOUBLE / grand_total ≤ 0.95  
            THEN 'B-tier'  
        ELSE 'C-tier'  
    END AS tier  
FROM ranked  
ORDER BY rn;
```

Interpretation:

- **A-tier** (top ~80% of volume): usually smooth demand — highest forecast value, standard methods work

- **B-tier** (next ~15%): transition zone — may need method switching
- **C-tier** (the tail): dominated by lumpy items — consider aggregating to category level or using intermittent demand methods

The strategic paradox: the items easiest to forecast (smooth, high-volume) are the ones where forecasting adds the least value. The items hardest to forecast (lumpy, low-volume) are where a good forecast would matter most.

Method Decision Flowchart

Use this one-page decision tree to select the right forecasting approach based on your data characteristics.

Step 1: Classify your demand pattern (Check 7)

```
Is ADI > 1.32?  
├─ NO → Is CV2 > 0.49?  
│   ├── NO → SMOOTH  
│   └── YES → ERRATIC  
└─ YES → Is CV2 > 0.49?  
    ├── NO → INTERMITTENT  
    └── YES → LUMPY
```

Step 2: Select method

DEMAND PATTERN	RECOMMENDED METHOD	FALLBACK	NOTES
Smooth	ETS / ARIMA / DeepAR	Prophet	Standard methods work well; neural forecasters justified at scale
Erratic	ARIMA	ETS with damped trend	Watch for outlier sensitivity; consider robust variants
Intermittent	Croston's method	SBA	Separate forecasts for interval and size
Lumpy	SBA / ADIDA	Aggregate to category	Temporally aggregate to reduce zeros, then disaggregate

Step 3: Consider aggregation

IF...	THEN...
> 50% of series are lumpy	Forecast weekly instead of daily
> 66% of series are lumpy	Forecast at category level, allocate downward
Activity rate < 0.05	Series is essentially dormant — use simple rules (e.g., reorder point)

Quick Reference Card

CHECK	WHAT	PASS CRITERIA	FAIL ACTION
1. Duplicates	(date, store_id, item_id) uniqueness	0 duplicates	Fix export query; deduplicate
2. Negatives	Negative demand or discounts	< 0.1% of rows	Investigate with POS team
3. Promos	Promo flag vs. discount consistency	Promos have discounts	Drop or enrich promo feature
4. Pricing	price - discount $\approx$ net_price	Error < 0.01 per row	Fix pricing pipeline
5. Price Spread	Within-day price variance	rel_range < 0.20	Aggregate to volume-weighted price
6. Store Coverage	All stores report daily	Coverage > 80%	Exclude ghost stores
7. Intermittency	SBC classification	< 50% lumpy	Switch to intermittent methods
8. Pareto	Demand concentration	80/20 rule holds	Tier your forecasting strategy

Questions or issues? Refer to the [full blog post \(https://kozlov.ski/posts/demand-forecast-eda\)](https://kozlov.ski/posts/demand-forecast-eda) for detailed explanations, visualizations, and strategic recommendations.